Cyclic Rotation

```
FUNCTION solution (A, K)
    RETURN A if empty
    REPEAT K TIMES
        REMOVE the last element from K
        STORE the removed element in temp
        INSERT temp at the beginning of A
    END REPEAT
END FUNCTION
```

Odd Occurrences In Array

```
FUNCTION solution (A)
    unpaired ← 0

    FOR EACH num IN A
        unpaired ← unpaired XOR num
    END FOR

    RETURN unpaired
END FUNCTION
```

Frog Jmp

```
FUNCTION solution (X, Y, D):
    CALCULATE the distance between the starting position and target
    ADD jump distance − 1 to calculated distance
    DIVIDE result by jump distance using floor division

    RETURN number of required jumps

END FUNCTION
```

Perm Missing Elem

```
FUNCTION solution (A):
    total_nums ← LENGTH(A) + 1
    predicted_total ← Sum of all integers from 1 to # of elements using Gauss summation
    total ← 0

    FOR EACH n IN A
        total ← total + n
    END FOR

    missing_num ← predicted_total − total
    RETURN missing_num

END FUNCTION
```

Tape Equilibrium

```
FUNCTION solution(A)
    left_sum ← first element of A
    right_sum ← 0

    FOR each remaining element in A:
        ADD the element to right_sum
    END FOR

    CALCULATE the absolute difference between left_sum and right_sum
    smallest_diff ← this calculated difference

    FOR EACH element from the second to the second-last element of A:
        ADD the current element to left_sum
        SUBTRACT the current element from right_sum
        CALCULATE the absolute difference between left_sum and right_sum

        IF new difference < smallest_diff:
            smallest_diff ← new difference
        END IF
    END FOR

    RETURN smallest_diff

END FUNCTION
```